

UrbanHub

Smart Housing Society Management Platform

Project Proposal Document

Submitted by

Arnav Narula — 2547115

Kartik Dewnani — 2547139

Ananya S Shetty — 2547161

Abstract

Housing societies and residential apartment complexes across India continue to rely on fragmented, manual systems such as WhatsApp groups, paper registers, and spreadsheets to manage day-to-day operations including complaint resolution, maintenance fee collection, visitor entry, and community communication. This lack of a unified digital system leads to delayed issue resolution, poor financial transparency, weak security at entry points, and reduced trust between residents and management committees. UrbanHub is a mobile-first society management application designed to digitize and streamline these processes through a single, role-based platform serving residents, committee members, and security personnel. The system enables residents to raise and track complaints in real time, make maintenance payments digitally, approve visitor and delivery entries remotely, and stay informed through instant announcements. The platform further distinguishes itself through artificial-intelligence-assisted complaint triage, real-time push-notification-driven visitor approval, and a transparent expense-reporting dashboard. By replacing ad-hoc manual processes with a structured digital workflow, UrbanHub aims to improve operational efficiency, security, transparency, and overall quality of life within residential communities.

Introduction

Motivation

The rapid growth of urban housing in India has led to a sharp increase in the number of gated communities and apartment complexes, many of which house hundreds of residents. Despite this growth, the tools used to manage these communities have remained largely manual and outdated. Residents typically report maintenance issues through phone calls or WhatsApp messages that are easily lost or forgotten. Maintenance fee collection is tracked manually in spreadsheets, making it difficult for residents to verify how their contributions are being utilized. Visitor and delivery management at security gates is handled through paper registers, which are slow, error-prone, and offer little real security. These inefficiencies create friction in daily community life and reduce the overall trust between residents and the managing committee. The motivation behind UrbanHub stems from the need to bring these processes into a single, reliable, and transparent digital system that is accessible to every resident, regardless of their technical familiarity.

Problem Statement

Existing methods of housing society management are fragmented across multiple informal channels, resulting in the following core problems: complaints raised by residents are not tracked systematically and often go unresolved or unacknowledged; maintenance payments are collected and recorded manually, leading to a lack of transparency and frequent disputes; visitor and delivery entries at the gate are recorded on paper, offering no real-time verification or resident control over who enters the premises; and residents have little to no visibility into how society funds are spent each month. There is currently no single, affordable, and easy-to-use mobile application that addresses all of these problems together for small and mid-sized housing societies.

Objectives

1. To design and develop a mobile application that enables residents to raise, track, and resolve maintenance complaints in real time.
2. To implement a secure digital payment system for monthly maintenance dues with automated receipt generation.
3. To build a real-time visitor and delivery approval system that notifies residents instantly and allows one-tap approval or rejection.
4. To provide a centralized announcement and notice board for transparent communication between the committee and residents.
5. To incorporate artificial intelligence for automatic complaint categorization and priority assignment.
6. To ensure data security and privacy through role-based access control and secure authentication.

Existing Works

Several mobile applications currently address aspects of housing society management. MyGate is one of the most widely adopted platforms in India, offering visitor management, security features, and community communication tools; however, it is primarily designed for large gated communities and can be cost-prohibitive for smaller societies. NoBrokerHood provides similar visitor management and maintenance tracking functionality but is tightly integrated with the broader NoBroker real-estate ecosystem, which may not suit all users. ApnaComplex offers accounting and complaint-tracking features but has been noted by users to have a relatively complex and dated interface. SocietyBook and Society Notebook provide lightweight, free

alternatives but lack advanced features such as AI-assisted automation or real-time visitor approval through push notifications. Across these existing solutions, a common limitation is that most are designed primarily around the needs of the management committee rather than the day-to-day resident experience, and few incorporate modern AI-based automation for complaint handling.

Benefits of the Proposed Work

- Provides a single unified platform for residents, committee members, and security staff, eliminating the need for multiple disconnected tools.
- Improves transparency through real-time complaint tracking, digital payment receipts, and visible expense breakdowns.
- Enhances residential security through photo-verified, push-notification-driven visitor approvals at the gate.
- Reduces the administrative burden on committee members through automated billing, reminders, and AI-assisted complaint triage.
- Improves resident satisfaction and community trust by making society operations transparent and easily accessible.
- Designed to be lightweight and cost-effective, making it suitable for small and mid-sized housing societies that are underserved by existing premium solutions.

Requirement Specification

Software Requirements

- Mobile Application Framework: React Native with Expo (cross-platform support for Android and iOS)
- Backend Framework: Node.js with Express.js for RESTful API development
- Database: PostgreSQL for relational data storage, managed through the Prisma ORM
- Caching and Session Management: Redis for OTP storage, rate limiting, and caching
- Authentication: JSON Web Token (JWT) based authentication with phone number and OTP verification
- Push Notifications: Expo Push Notification API for real-time alerts
- Cloud File Storage: Cloudflare R2 (S3-compatible) for storing complaint photographs and visitor images

- **Artificial Intelligence:** Google Gemini API for automated complaint categorization and priority assignment
- **Payment Gateway:** Razorpay or equivalent UPI-based payment gateway for maintenance fee collection
- **Development Tools:** Visual Studio Code, Git and GitHub for version control, Postman for API testing
- **Deployment Platforms:** Railway or Render for backend hosting, Neon for managed PostgreSQL hosting

Hardware Requirements

- **Development Machine:** A computer with a minimum of 8 GB RAM, a multi-core processor, and at least 20 GB of free storage for development tools and dependencies
- **Mobile Devices:** Android smartphones (Android 8.0 or higher) with a minimum of 2 GB RAM for application testing and deployment
- **Server Infrastructure:** Cloud-hosted virtual server instance with a minimum of 1 vCPU and 1 GB RAM for hosting the backend application and database in the pilot phase
- **Network Connectivity:** Stable internet connectivity for both development and deployment environments, supporting HTTPS-based API communication
- **Camera Module:** Functional camera on resident and security guard devices for capturing complaint photographs and visitor verification images

Plan of Work

The development of UrbanHub is organized into clearly defined modules, each assigned to a responsible team member with an estimated duration for completion. The plan below outlines the sequence of tasks from initial research through to final testing and deployment.

Task / Module	Person Responsible	Duration
Requirement gathering, market research, and competitor analysis	Arnav Narula	1 Week
System design — database schema, API design, and UI/UX wireframes	Kartik Dewnani	2 Weeks
User authentication module (OTP-based login and JWT session management)	Ananya S Shetty	1 Week
Complaint management module (raise, track, resolve, AI-based triage)	Arnav Narula	2 Weeks
Maintenance payment module (billing, UPI integration, digital receipts)	Kartik Dewnani	2 Weeks
Visitor and delivery approval module (gate registration, push-notification approval)	Ananya S Shetty	2 Weeks
Announcement and notice board module with read receipts	Arnav Narula	1 Week
Security implementation — role-based access control, rate limiting, data encryption	Kartik Dewnani	1 Week
Integration testing, bug fixing, and performance optimization	Ananya S Shetty	1.5 Weeks
Deployment, pilot rollout, and documentation	All Members	1.5 Weeks

Total estimated duration: approximately 15 weeks, from initial research through to pilot deployment.